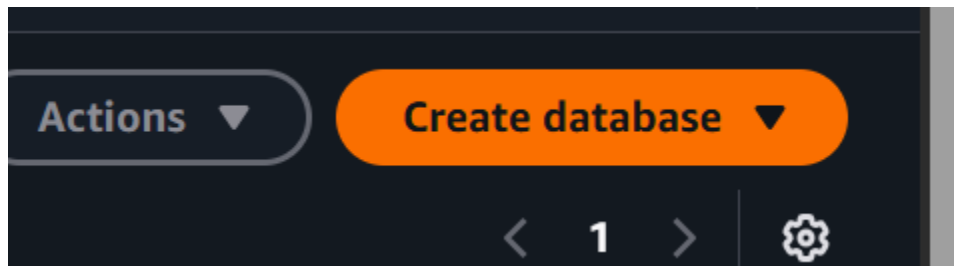
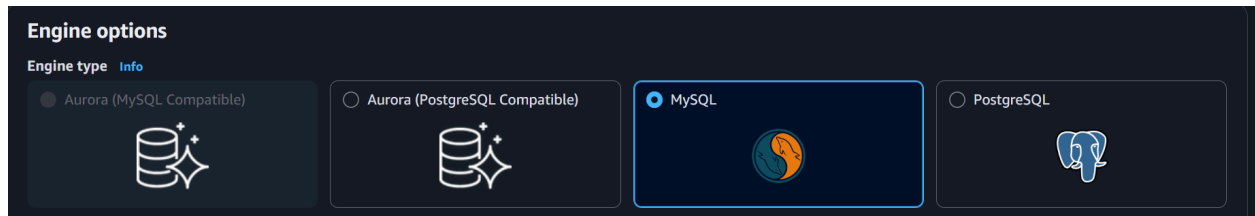


Now lets do the same thing using the RDS as data base

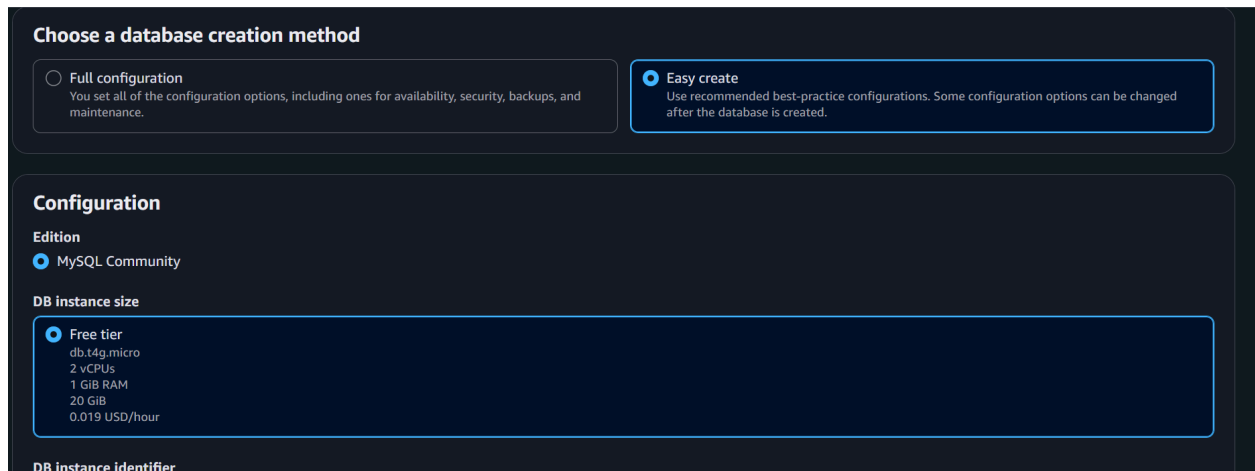
So lets create a RDS server



With full configuration



Now select your application



Enter your name and password

[Aurora and RDS](#) > [Databases](#) > Create database

Type a login ID for the master user of your DB instance.

root

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management

You can use AWS Secrets Manager or manage your master user credentials.

☐ Managed in AWS Secrets Manager - *most secure*
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ Self managed
Create your own password or have RDS generate one for you.

☐ Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password

Info

.....

Password strength **Very weak**

Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / ' " @

Confirm master password

Info

.....

And your DB

Databases (1) Group resources Modify Actions Create database

Filter by databases

DB identifier	Status	Role	Engine	Upgrade rollout order
database-1	Creating	Instance	MySQL Co...	SECOND

Now lets launch an server install everything

Install an ubuntu server with the 25 storage and `m7i-flex.large` as instance type

Instances (1/1) Info Connect Instance state Actions Launch instances

Saved filter sets
Choose filter set

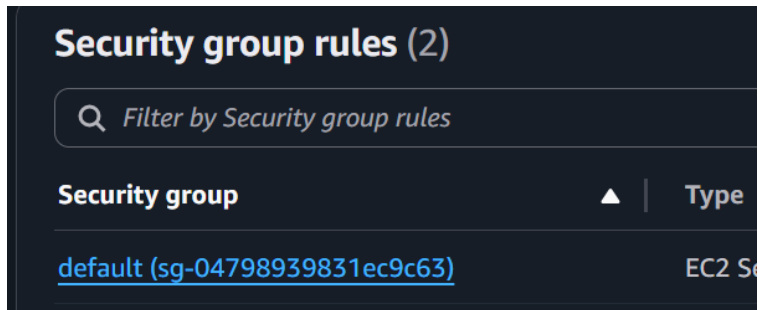
Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check
full-stack	i-058cacd33f9873c93	Terminated	m7i-flex.large	3/3 checks passed

Now if we want our server and database can communicate the

Your RDS must allow inbound traffic from your Ubuntu EC2.

- Go to **RDS** → **your DB** → **Connectivity & security**



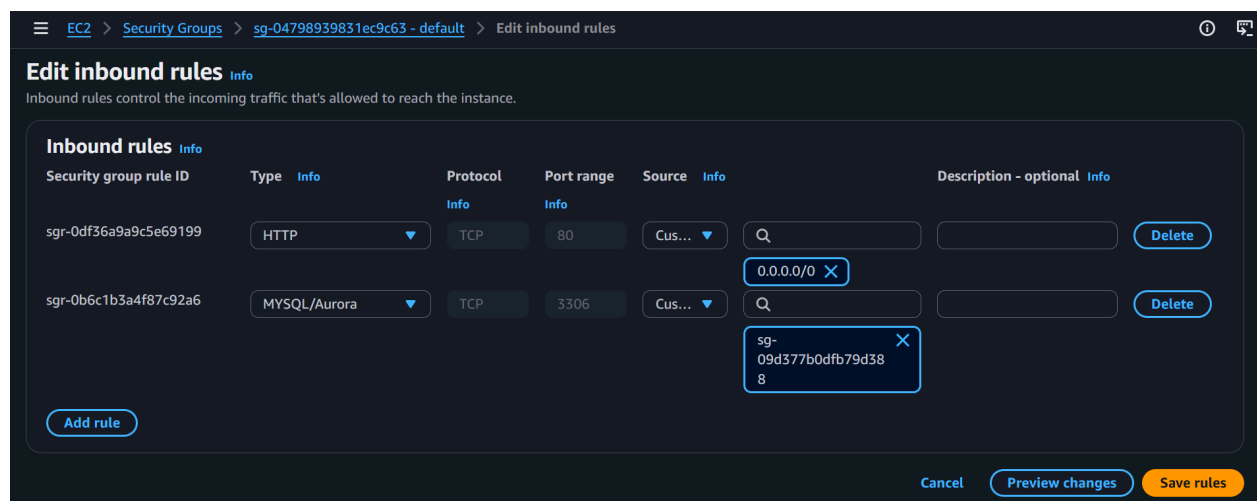
-
- Look at its **VPC security group**
- Edit inbound rules:

You need something like:

Type: MySQL/Aurora

Port: 3306

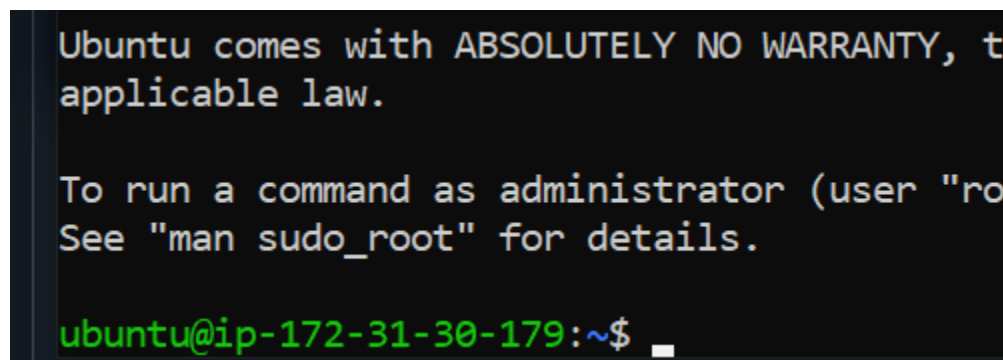
Source: <EC2 security group ID>



Add these and click on save

Now let's install everything

Now let's connect it



Now lets begin

Now lets install everything

As will be running the project of react springboot and mysql

- 1 sudo apt update
- 2 sudo apt install nodejs npm openjdk-21-jdk maven mysql* -y

```
60 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-30-179:~$ sudo apt install nodejs npm openjdk-21-jdk maven mysql* -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'mysql-testsuite' for glob 'mysql*'
Note, selecting 'mysql-server-5.5' for glob 'mysql*'
Note, selecting 'mysql-server-5.6' for glob 'mysql*'
Note, selecting 'mysql-server-5.7' for glob 'mysql*'
Note, selecting 'mysql-server-8.0' for glob 'mysql*'
Note, selecting 'mysql-client-5.5' for glob 'mysql*'
Note, selecting 'mysql-client-5.6' for glob 'mysql*'
```

- 3 nodejs -v
- 4 npm -v
- 5 mvn -n
- 6 mvn -v
- 7 java -v
- 8 java --version
- 9 mysql
- 10 ls

```
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
ubuntu@ip-172-31-30-179:~$ java --version
openjdk 21.0.10 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
ubuntu@ip-172-31-30-179:~$ npm -v
9.2.0
ubuntu@ip-172-31-30-179:~$ nodejs -v
v18.19.1
ubuntu@ip-172-31-30-179:~$ mvn --version
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 21.0.10, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.17.0-1007-aws", arch: "amd64", family: "unix"
ubuntu@ip-172-31-30-179:~$ mysql
ERROR 1045 (28000): Access denied for user 'ubuntu'@'localhost' (using password: NO)
ubuntu@ip-172-31-30-179:~$ mysql --version
mysql Ver 8.0.45-0ubuntu0.24.04.1 for Linux on x86_64 ((Ubuntu))
ubuntu@ip-172-31-30-179:~$
```

Now clone your repo

> git clone <https://github.com/mantu0tech/Fullstack-application.git>

>cd Fullstack-application/

>git checkout Design

```

ubuntu@ip-172-31-30-179:~$ ls
ubuntu@ip-172-31-30-179:~$ git clone https://github.com/mantu0tech/Fullstack-application.git
Cloning into 'Fullstack-application'...
remote: Enumerating objects: 115, done.
remote: Counting objects: 100% (115/115), done.
remote: Compressing objects: 100% (88/88), done.
remote: Total 115 (delta 24), reused 103 (delta 16), pack-reused 0 (from 0)
Receiving objects: 100% (115/115), 249.72 KiB | 19.21 MiB/s, done.
Resolving deltas: 100% (24/24), done.
ubuntu@ip-172-31-30-179:~$ ls
Fullstack-application
ubuntu@ip-172-31-30-179:~$ cd Fullstack-application/
ubuntu@ip-172-31-30-179:~/Fullstack-application$ ls
Backend  Frontend  db.sql
ubuntu@ip-172-31-30-179:~/Fullstack-application$ git branch
* main
ubuntu@ip-172-31-30-179:~/Fullstack-application$ git checkout Design
branch 'Design' set up to track 'origin/Design'.
Switched to a new branch 'Design'

```

Now lets try to connect your DB

If its shows available then only we can connect

Successfully created database database-1
[View connection details](#)

You can use settings from database-1 to simplify configuration of **suggested database add-ons** while we finish creating your DB for you.

Databases (1)

Group resources

Modify

Actions

Create database

DB identifier	Status	Role	Engine	Upgrade rollout order
database-1	Available	Instance	MySQL Co...	SECOND

If its working then only we can move forward

mysql -h database-1.csd8s4wqyqd0.us-east-1.rds.amazonaws.com -P 3306 -u root -p

```

ubuntu@ip-172-31-30-179:~$ ls
Fullstack-application
ubuntu@ip-172-31-30-179:~$ mysql -h database-1.csd8s4wqyqd0.us-east-1.rds.amazonaws.com -P 3306 -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 32
Server version: 8.4.8 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```

Now we need to execute the database.sql file in to our sql

```

backend - Frondend db.sql
ubuntu@ip-172-31-30-179:~/Fullstack-application$ mysql -h database-1.csd8s4wqyqd0.us-east-1.rds.amazonaws.com -P 3306 -
u root -p <db.sql
Enter password:

```

And its done

```
mysql -h database-1.csd8s4wqyqd0.us-east-1.rds.amazonaws.com -P 3306 -u root -p <db.sql
```

Now our db is up and running now lets change the code and make to open it for the db and backend

Now change the local host of your server to EC2 public ip

```

26 ls
27 cd src/main/resources/
28 ls
29 vi application.properties
30 cd ..
31 cd Frondend/
32 ls
33 ls -la
34 vi .env
35 ls

```

Add the DB ulr or hostname here in the file

Add these link here

```
>>spring.datasource.url=jdbc:mysql://database-1.csd8s4wqyqd0.us-east-1.rds.amazonaws.com:3306/ClickNcart
```

```

spring.application.name=ClickNcart
spring.datasource.url=jdbc:mysql://database-1.csd8s4wqyqd0.us-east-1.rds.amazonaws.com:3306/ClickNcart
spring.datasource.username=root
spring.datasource.password=rootroot
spring.jpa.hibernate.ddl-auto=update
#optional
spring.jpa.show-sql=true
spring.jpa.properties.format_sql=true
#spring.web.error.include-message=always
server.error.include-message=always

spring.data.rest.base-path=/api
razorpay.key_id=rzp_test_SXLPyp2eitE0Px
razorpay.key_secret=6w8f0gbfMZyNat6YH7FDFtUM

origin=http://34.229.198.138:3000
~
~
~
~
~
~
~
~

```

Now lets run our backend

```
> mvn clean install
```

```

README.md  mvnw  mvnw.cmd  pom.xml  src
ubuntu@ip-172-31-30-179:~/Fullstack-application/Backend$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:ClickNcart >-----
[INFO] Building ClickNcart 0.0.1-SNAPSHOT
[INFO] -----[ jar ]-----

```

Now lets verify it

```

[INFO] -----
ubuntu@ip-172-31-30-179:~/Fullstack-application/Backend$ ls
README.md  mvnw  mvnw.cmd  pom.xml  src  target
ubuntu@ip-172-31-30-179:~/Fullstack-application/Backend$ cd target/
ubuntu@ip-172-31-30-179:~/Fullstack-application/Backend/target$ ls
ClickNcart-0.0.1-SNAPSHOT.jar          classes          generated-test-sources  maven-status      test-classes
ClickNcart-0.0.1-SNAPSHOT.jar.original  generated-sources  maven-archiver          surefire-reports
ubuntu@ip-172-31-30-179:~/Fullstack-application/Backend/target$

```

Now lets run it

>> java -jar ClickNcart-0.0.1-SNAPSHOT.jar

```

ClickNcart-0.0.1-SNAPSHOT.jar.original  generated-sources  maven-archiver          surefire-reports
ubuntu@ip-172-31-30-179:~/Fullstack-application/Backend/target$ java -jar ClickNcart-0.0.1-SNAPSHOT.jar

```

Now run our frontend

```

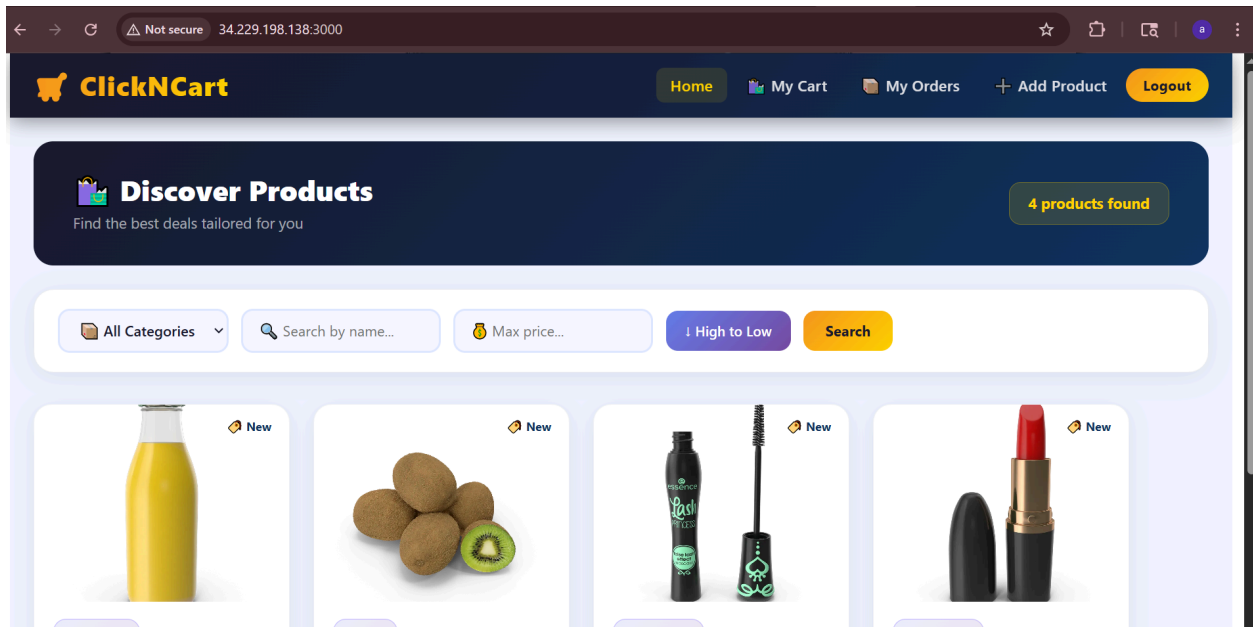
Line 97:6:  React Hook useEffect has a missing dependency: 'params.id'. Either include it or remove the dependency &
ay  react-hooks/exhaustive-deps

src/components/Register.js
  Line 102:45:  'errors' is assigned a value but never used  no-unused-vars
  Line 124:23:  Expected '===' and instead saw '=='                        egeqeq

src/components/UpdateProduct.js
  Line 2:10:  'NavLink' is defined but never used
           no-unused-vars
  Line 84:6:  React Hook useEffect has a missing dependency: 'params.id'. Either include it or remove the dependency &
ay  react-hooks/exhaustive-deps

webpack compiled with 1 warning

```



And we are done!!
Now lets delete everything